

```

# This Python 3 environment comes with many helpful analytics
libraries installed
# It is defined by the kaggle/python Docker image:
https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/"
directory
# For example, running this (by clicking run or pressing Shift+Enter)
will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/)
that gets preserved as output when you create a version using "Save &
Run All"
# You can also write temporary files to /kaggle/temp/, but they won't
be saved outside of the current session

/kaggle/input/titanic/train.csv
/kaggle/input/titanic/test.csv
/kaggle/input/titanic/gender_submission.csv

```

This below code block performs the initial data loading and preparation for the Titanic dataset. It reads the training and testing data from CSV files into Pandas DataFrames (**df_train** and **df_test**, respectively). A new column, **train_test**, is created to distinguish between the two datasets (1 for training, 0 for testing). To facilitate combined preprocessing, the two DataFrames are concatenated into a single DataFrame called **all_data**. Before concatenation, a **Survived** column with NaN values is added to the test set to ensure consistent column structure. Finally, the code displays the first 5 rows of the training dataset using **.head(5)** and the columns present in the combined dataset using **.columns**. The **%matplotlib** inline magic command enables the display of matplotlib plots within the Jupyter Notebook output.

```

#Importing the Training and Testing datasets to the dataframes
'df_train' and 'df_test'
df_train = pd.read_csv('/kaggle/input/titanic/train.csv')
df_test = pd.read_csv('/kaggle/input/titanic/test.csv')
df_train['train_test'] = 1
df_test['train_test'] = 0
#Concat the Training and Testing datasets to 'all_data'
df_test['Survived'] = np.NaN
all_data = pd.concat([df_train, df_test])

```

```
%matplotlib inline
all_data.columns
#Printing and observing the first 5 rows of the training dataset
df_train.head(5)
```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	

		Name	Sex	Age
SibSp	\			
0		Braund, Mr. Owen Harris	male	22.0
1				
1	Cummings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	
1				
2	Heikkinen, Miss. Laina	female	26.0	
0				
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	
1				
4	Allen, Mr. William Henry	male	35.0	
0				

	Parch	Ticket	Fare	Cabin	Embarked	train_test
0	0	A/5 21171	7.2500	NaN	S	1
1	0	PC 17599	71.2833	C85	C	1
2	0	STON/O2. 3101282	7.9250	NaN	S	1
3	0	113803	53.1000	C123	S	1
4	0	373450	8.0500	NaN	S	1

```
#Understanding the distribution of numeric training data
df_train.describe()
```

	PassengerId	Survived	Pclass	Age	SibSp	\
count	891.000000	891.000000	891.000000	714.000000	891.000000	
mean	446.000000	0.383838	2.308642	29.699118	0.523008	
std	257.353842	0.486592	0.836071	14.526497	1.102743	
min	1.000000	0.000000	1.000000	0.420000	0.000000	
25%	223.500000	0.000000	2.000000	20.125000	0.000000	
50%	446.000000	0.000000	3.000000	28.000000	0.000000	
75%	668.500000	1.000000	3.000000	38.000000	1.000000	
max	891.000000	1.000000	3.000000	80.000000	8.000000	

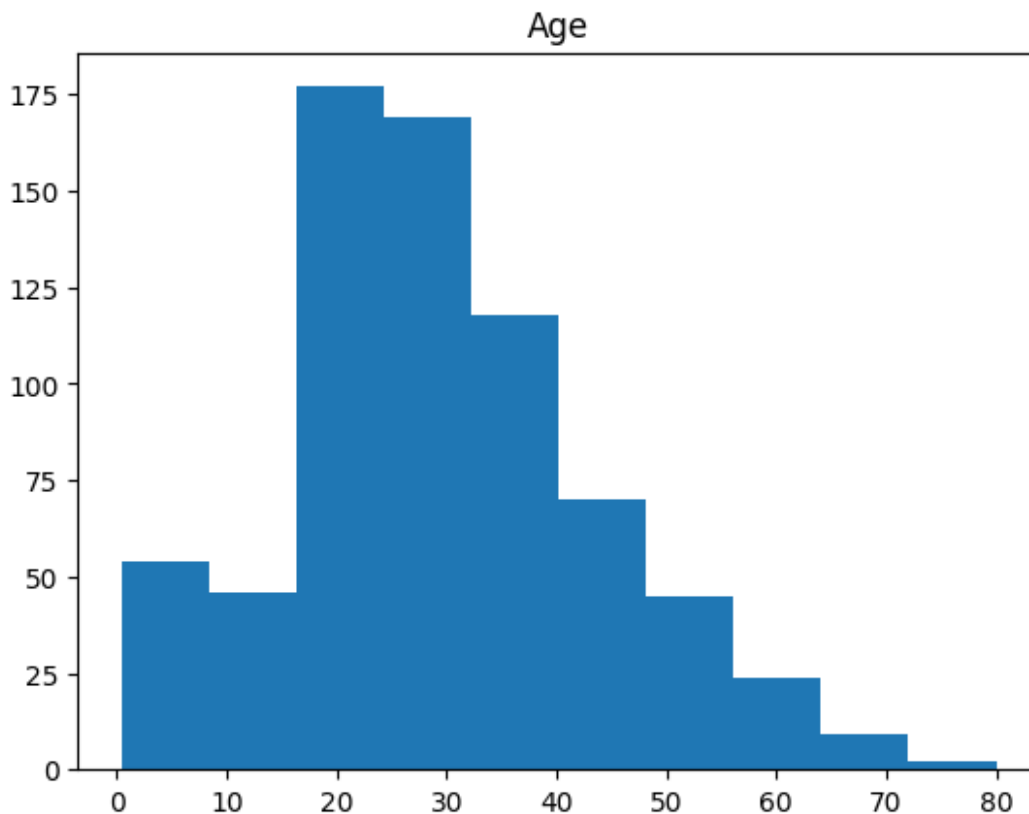
	Parch	Fare	train_test
count	891.000000	891.000000	891.0
mean	0.381594	32.204208	1.0
std	0.806057	49.693429	0.0
min	0.000000	0.000000	1.0

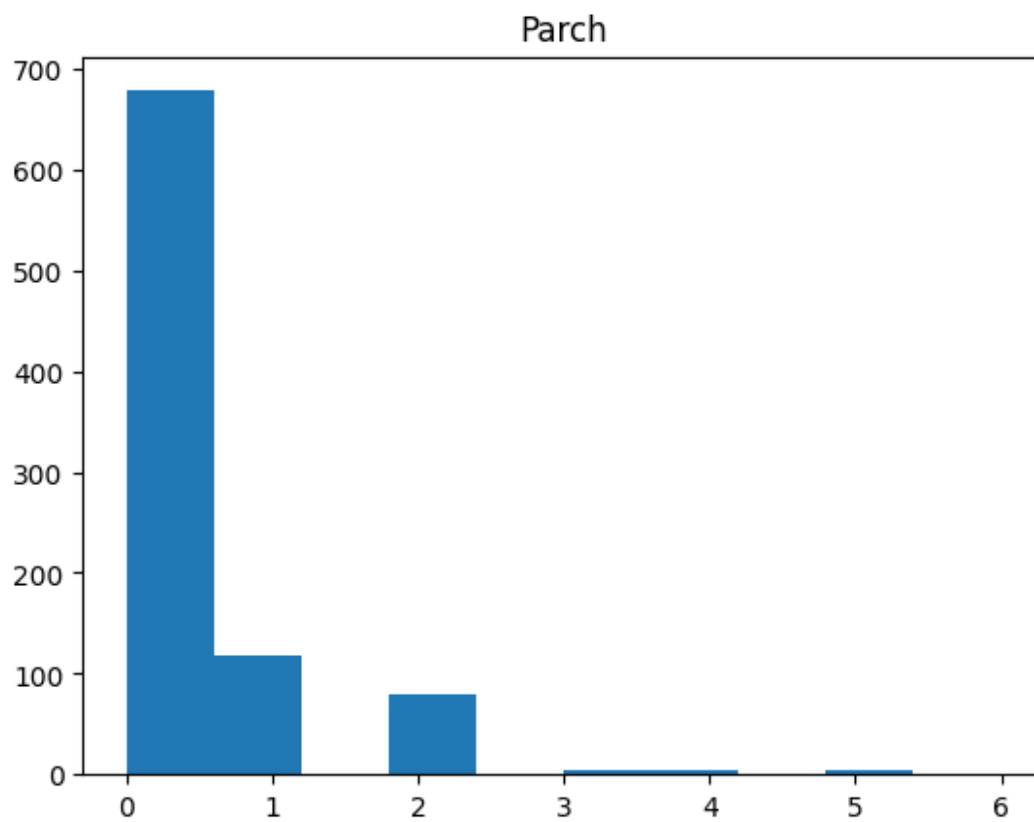
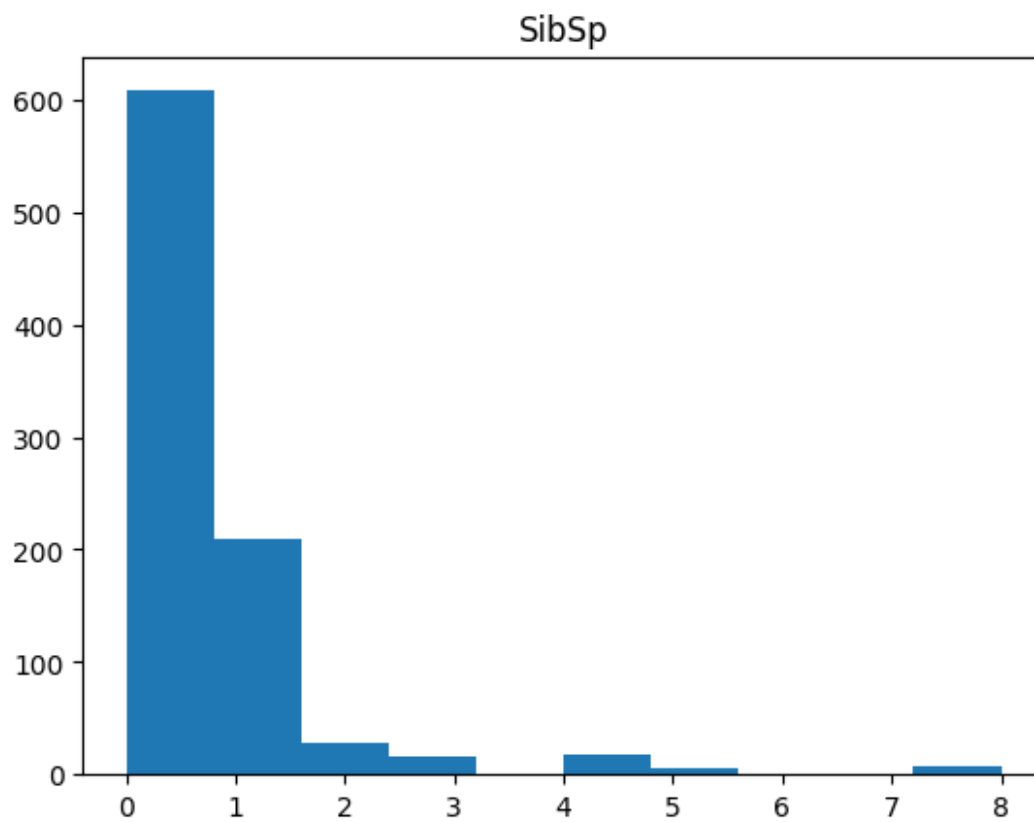
25%	0.000000	7.910400	1.0
50%	0.000000	14.454200	1.0
75%	0.000000	31.000000	1.0
max	6.000000	512.329200	1.0

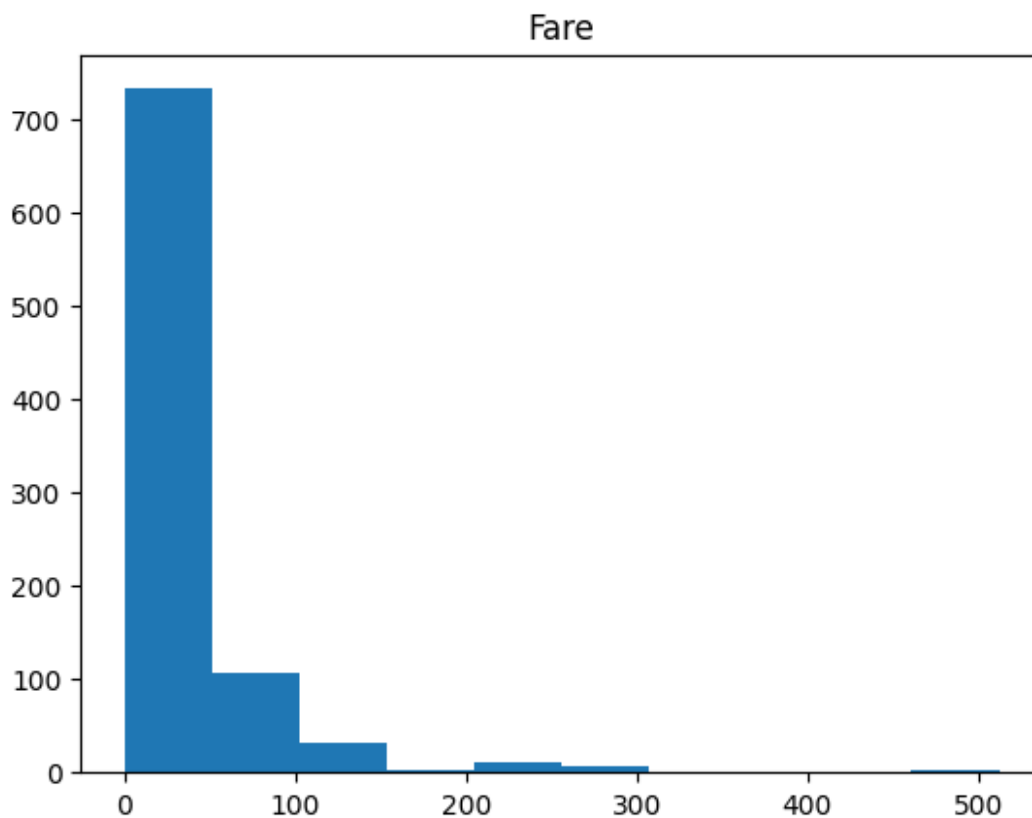
```
#Creating two separate dataframes from the training data
#One to store the numerical columns(df_num) and another to store the
categorical columns(df_cat)
df_num = df_train[['Age','SibSp','Parch','Fare']]
df_cat =
df_train[['Survived','Pclass','Sex','Ticket','Cabin','Embarked']]

#Plotting histograms of all the columns of df_num to understand the
data distribution
import matplotlib.pyplot as plt

for i in df_num.columns:
    plt.hist(df_num[i])
    plt.title(i)
    plt.show()
```



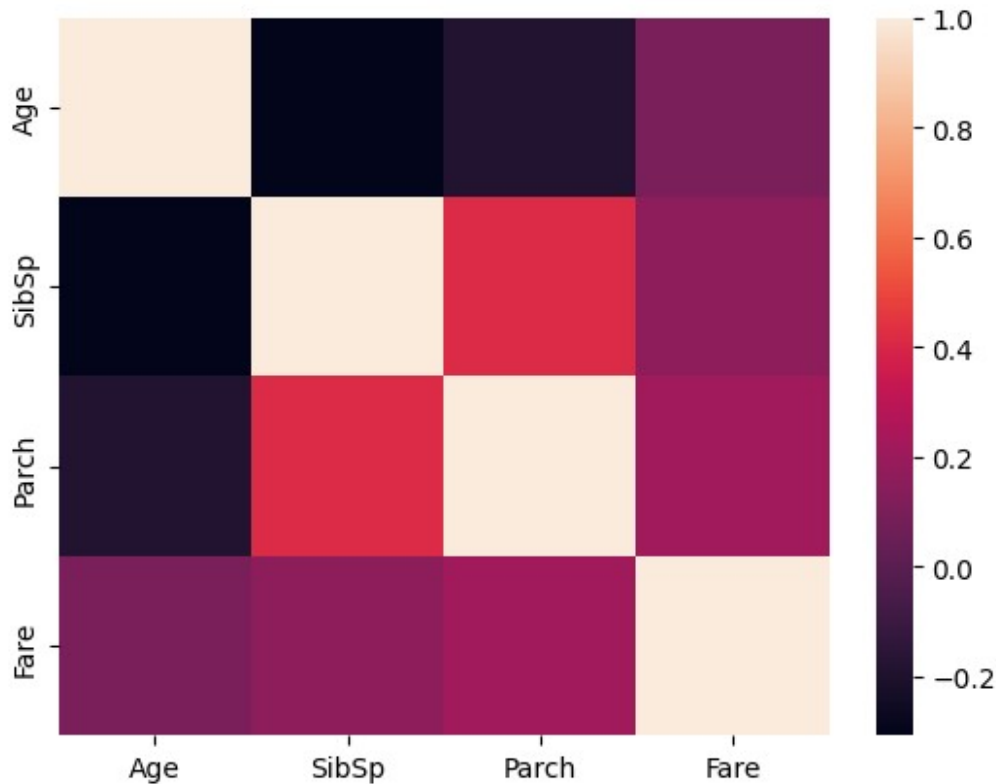




```
#Ploting heatmap of all the columns of df_num to understand the correlation between these variables  
import seaborn as sns  
print(df_num.corr())  
sns.heatmap(df_num.corr())
```

	Age	SibSp	Parch	Fare
Age	1.000000	-0.308247	-0.189119	0.096067
SibSp	-0.308247	1.000000	0.414838	0.159651
Parch	-0.189119	0.414838	1.000000	0.216225
Fare	0.096067	0.159651	0.216225	1.000000

<Axes: >



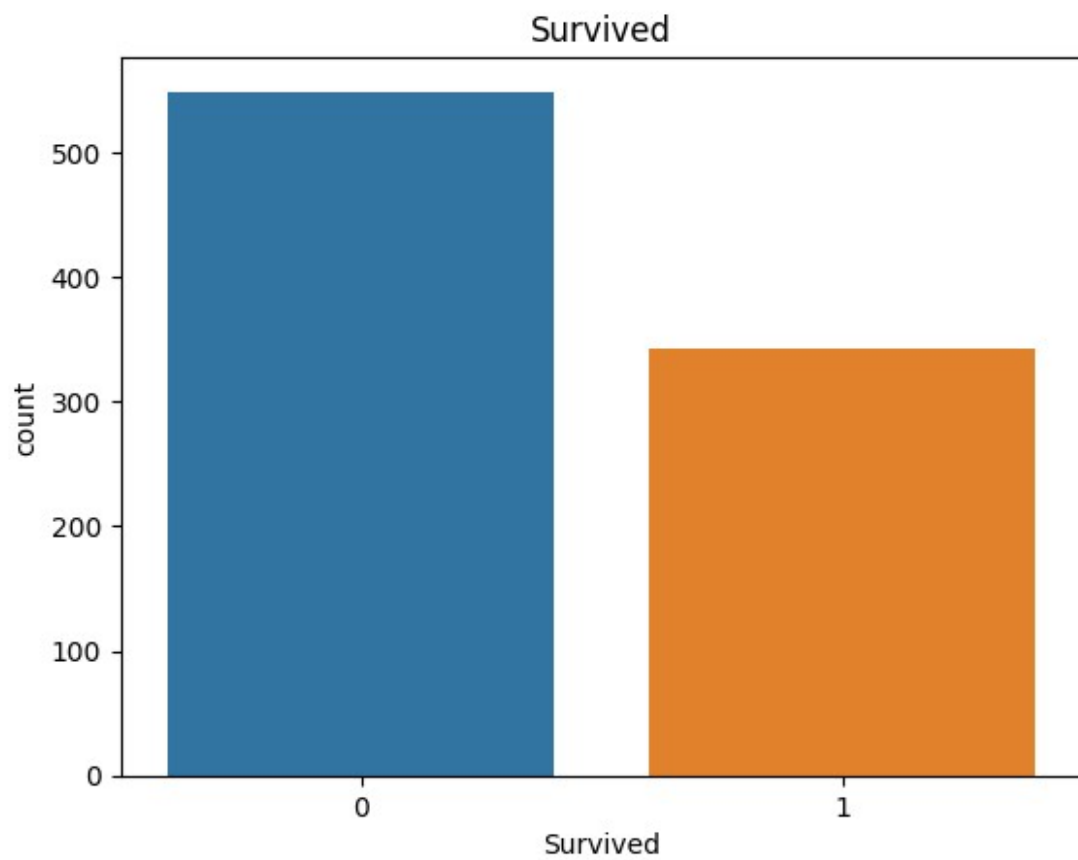
```
#Plotting the pivot table that shows average value of the variable for
each outcome
pd.pivot_table(df_train, index='Survived', values = ['Age', 'SibSp',
'Parch', 'Fare'])
```

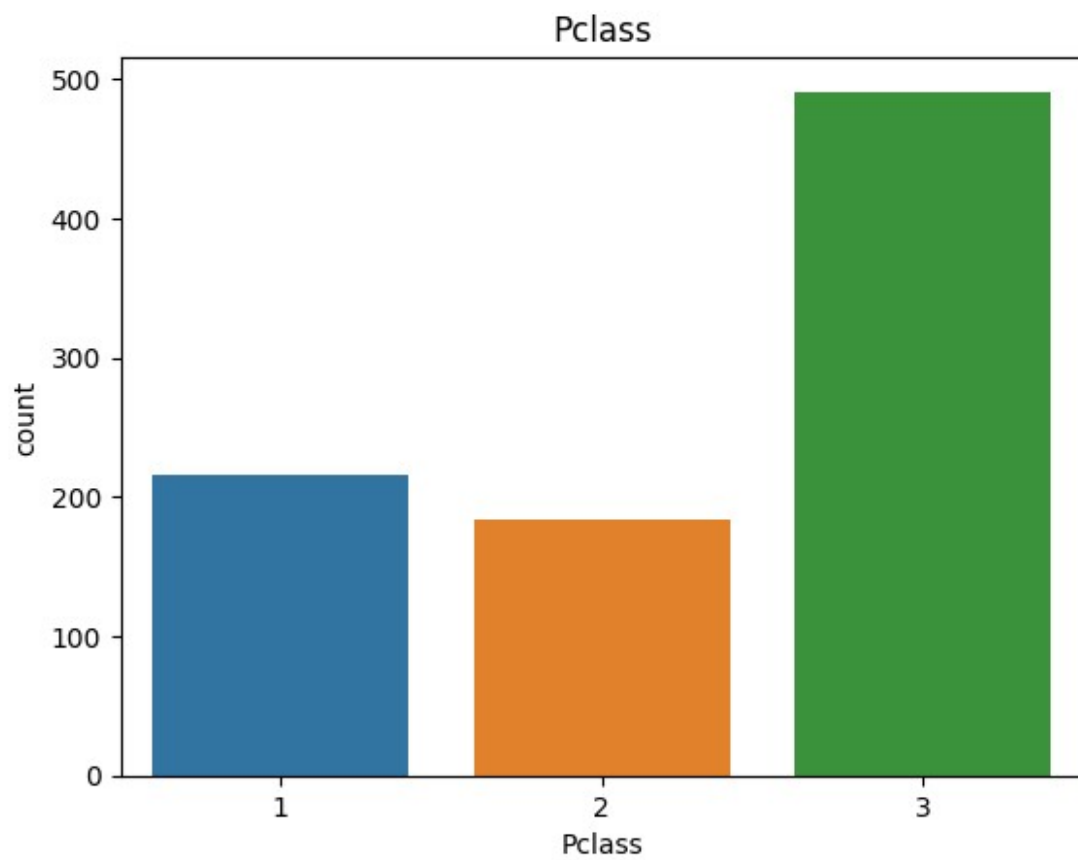
	Age	Fare	Parch	SibSp
Survived				
0	30.626179	22.117887	0.329690	0.553734
1	28.343690	48.395408	0.464912	0.473684

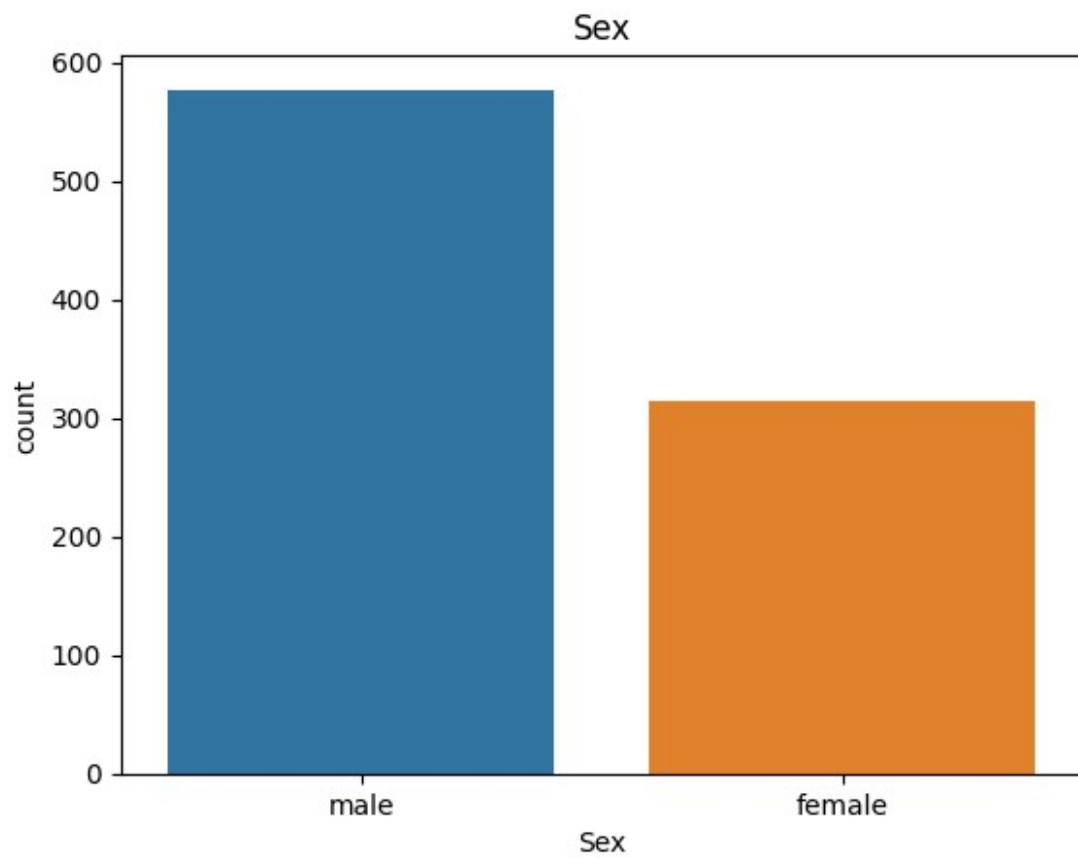
It is clear from the above table that the value of the outcome is independant of any specific value or range of values of the numerical columns/data.

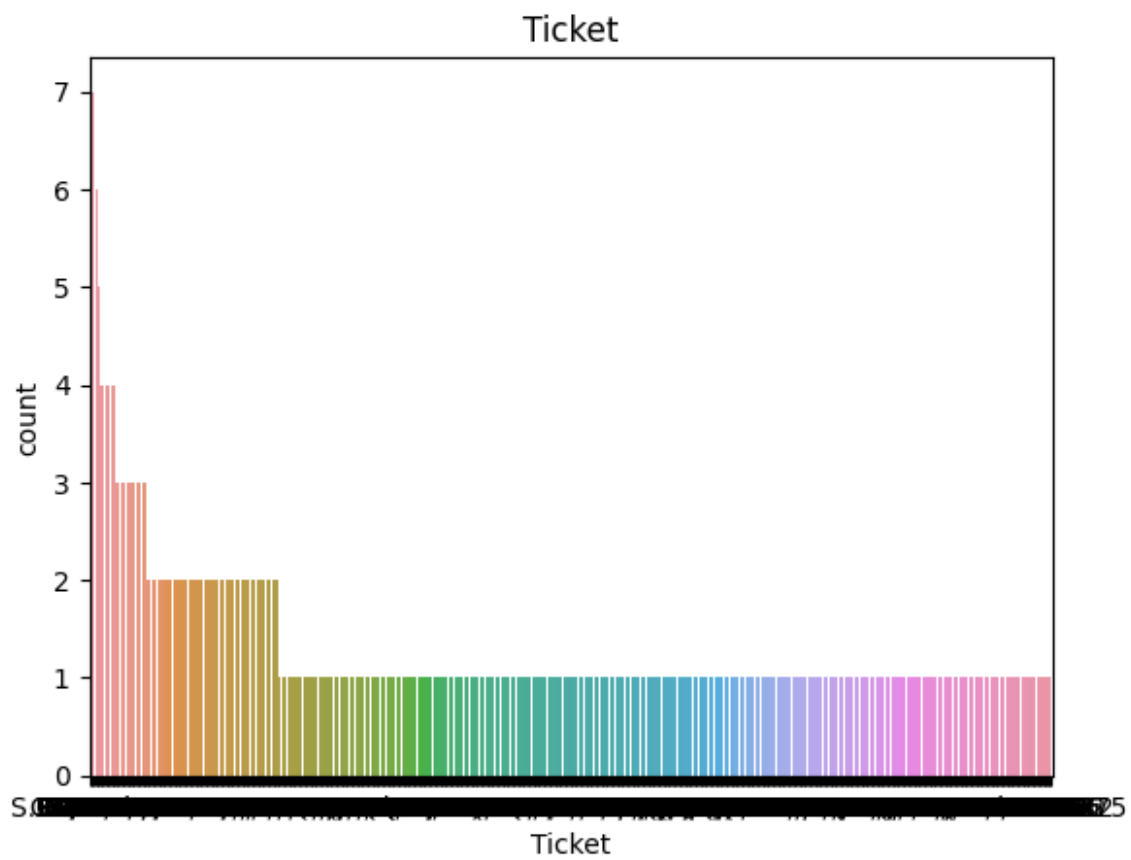
There's one exception, the 'Fare' column. Here, it is clear that those who paid a higher amount of Fare, survived.

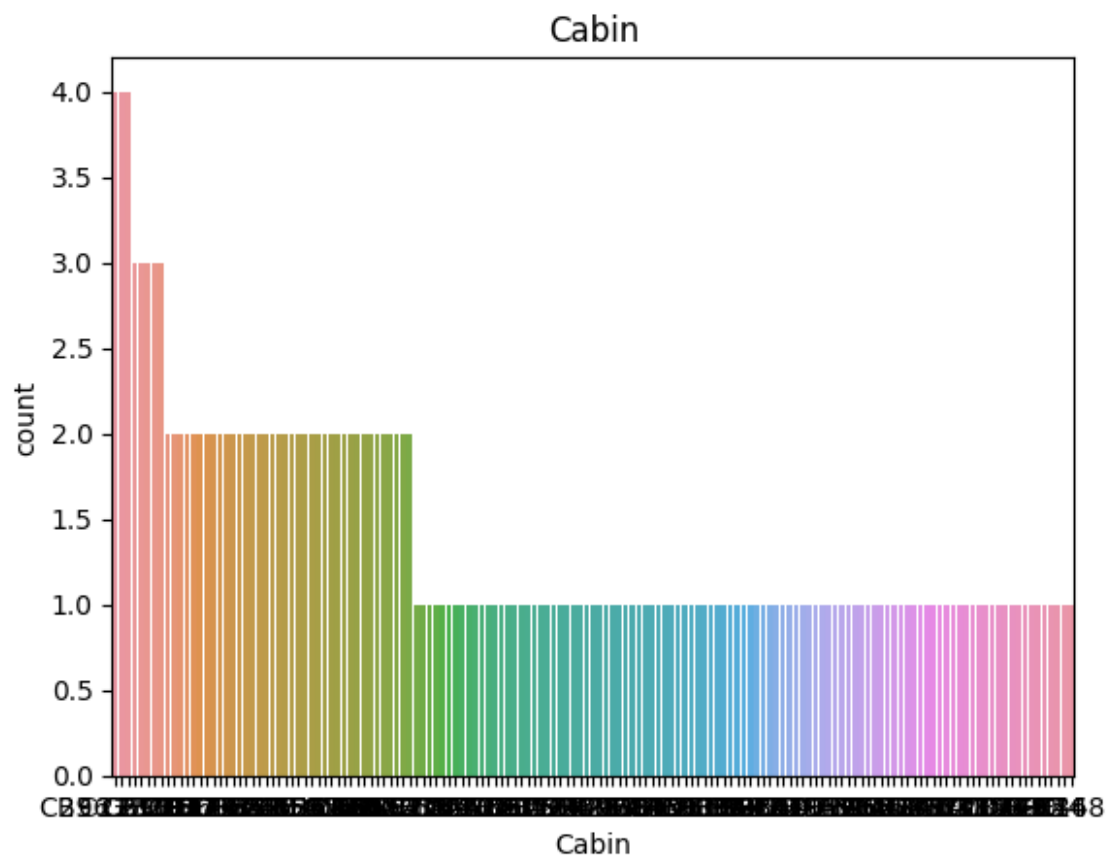
```
#Plotting categorical columns to understand the categorical data
distribution
for i in df_cat.columns:
    sns.barplot(x=df_cat[i].value_counts().index,
y=df_cat[i].value_counts()).set_title(i)
plt.show()
```

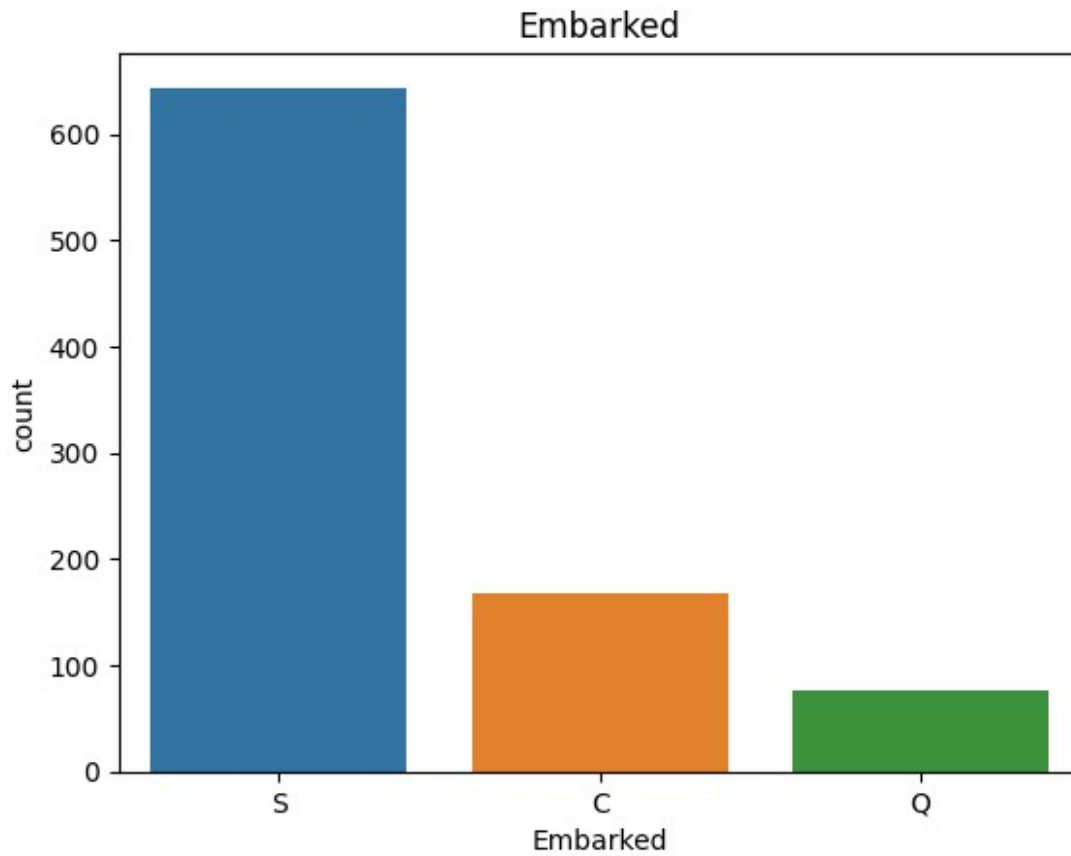












#Creating Pivot Tables using the categorical data to collect insights if any

```
print(pd.pivot_table(df_train, index='Survived', columns = 'Pclass',
values='Ticket', aggfunc = 'count'))
print(pd.pivot_table(df_train, index='Survived', columns = 'Sex',
values='Ticket', aggfunc = 'count'))
print(pd.pivot_table(df_train, index='Survived', columns = 'Embarked',
values='Ticket', aggfunc = 'count'))
```

Pclass	1	2	3
Survived			
0	80	97	372
1	136	87	119
Sex	female	male	
Survived			
0	81	468	
1	233	109	
Embarked	C	Q	S
Survived			
0	75	47	427
1	93	30	217

Following insights can be drawn out of the above three Pivot Tables.

- Class 1 passengers have the highest percentage of survival, whereas the Class 3 passengers have the lowest percentage of survival.
- Female passengers have the highest percentage of survival, whereas the Male passengers have the lowest percentage of survival.
- Impact of the Port of Embarkation on survival is not very clear.

Feature Engineering

1. Simplyfy cabins
2. Do different ticket types impact survival rate?
3. Does a person's title relate to survival rate?

```
df_cat.Cabin
df_train['cabin_multiple'] = df_train.Cabin.apply(lambda x: 0 if
pd.isna(x) else len(x.split(' ')))
df_train['cabin_multiple'].value_counts()
```

```
cabin_multiple
```

```
0    687
1    180
2     16
3      6
4      2
```

```
Name: count, dtype: int64
```

#Pivot Table: cabin_multiple vs Survived

```
pd.pivot_table(df_train, index = 'Survived', columns =
'cabin_multiple', values='Ticket', aggfunc='count')
```

cabin_multiple	0	1	2	3	4
Survived					
0	481.0	58.0	7.0	3.0	NaN
1	206.0	122.0	9.0	3.0	2.0

Impact of the cabin_multiple on survival is not very clear.

```
df_train['cabin_adv'] = df_train.Cabin.apply(lambda x: str(x)[0])
```

#Pivot Table: To understand the impact of different types of Cabins on survival

```
print(df_train.cabin_adv.value_counts())
```

```
pd.pivot_table(df_train, index='Survived', columns= 'cabin_adv',
values= 'Name', aggfunc= 'count')
```

```
cabin_adv
```

```
n    687
C     59
B     47
D     33
E     32
A     15
```

```
F      13
G       4
T       1
Name: count, dtype: int64
```

```
cabin_adv      A      B      C      D      E      F      G      T      n
Survived
0           8.0    12.0    24.0     8.0     8.0     5.0     2.0     1.0    481.0
1           7.0    35.0    35.0    25.0    24.0     8.0     2.0    NaN    206.0
```

```
df_train.Name.head(50)
df_train['name_title'] = df_train.Name.apply(lambda x: x.split(',')[1].split('.')[0].strip())
```

```
df_train['name_title'].value_counts()
```

```
name_title
Mr          517
Miss        182
Mrs         125
Master       40
Dr           7
Rev          6
Mlle         2
Major        2
Col          2
the Countess 1
Capt        1
Ms           1
Sir          1
Lady         1
Mme          1
Don          1
Jonkheer     1
Name: count, dtype: int64
```

```
df_train['numeric_ticket'] = df_train.Ticket.apply(lambda x: 1 if x.isnumeric() else 0)
df_train['ticket_letter'] = df_train.Ticket.apply(lambda x: ''.join(x.split(' ')[::-1]).replace('.', '').replace('/', '').lower() if len(x.split(' ')[::-1])>0 else 0)
```

```
df_train['numeric_ticket'].value_counts()
```

```
numeric_ticket
1      661
0      230
Name: count, dtype: int64
```

```
#pd.set_option("max_rows", None)
df_train['ticket_letter'].value_counts()
```

ticket_letter	
0	665
pc	60
ca	41
a5	21
stono2	18
sotonog	15
scparis	11
wc	10
a4	7
soc	6
fcc	5
c	5
sopp	3
pp	3
wep	3
ppp	2
scah	2
sotono2	2
swpp	2
fc	1
scahbasle	1
as	1
sp	1
sc	1
scow	1
fa	1
sop	1
sca4	1
casoton	1

Name: count, dtype: int64

Data Pre-processing

1. Drop null values from Embarked (only 2)
2. Include only relevant variables
3. Do categorical transforms on all data.
4. Impute data with mean for fare and age.
5. Normalize fare using log to give more semblance of a normal distribution
6. Scale data 0-1 using standard scaler

```

all_data['cabin_multiple'] = all_data.Cabin.apply(lambda x: 0 if
pd.isna(x) else len(x.split(' ')))
all_data['cabin_adv'] = all_data.Cabin.apply(lambda x: str(x)[0])
all_data['name_title'] = all_data.Name.apply(lambda x: x.split(',')[
1].split('.')[0].strip())
all_data['numeric_ticket'] = all_data.Ticket.apply(lambda x: 1 if
x.isnumeric() else 0)
all_data['ticket_letter'] = all_data.Ticket.apply(lambda x:

```

```

''.join(x.split(' ')[:-1]).replace('.', '').replace('/', '').lower() if
len(x.split(' ')[:-1])>0 else 0)

#impute nulls for continuous data
all_data.Age = all_data.Age.fillna(df_train.Age.mean())
all_data.Fare = all_data.Fare.fillna(df_train.Fare.mean())

#drop null Embarked rows
all_data.dropna(subset = 'Embarked', inplace = True)

#log norm of sibsp
all_data['norm_sibsp'] = np.log(all_data.SibSp+1)
all_data['norm_sibsp'].hist()

#log norm of fare
all_data['norm_fare'] = np.log(all_data.Fare+1)
all_data['norm_fare'].hist()

#converted Fare to Category for pd.get_dummies()
all_data.Pclass = all_data.Pclass.astype(str)

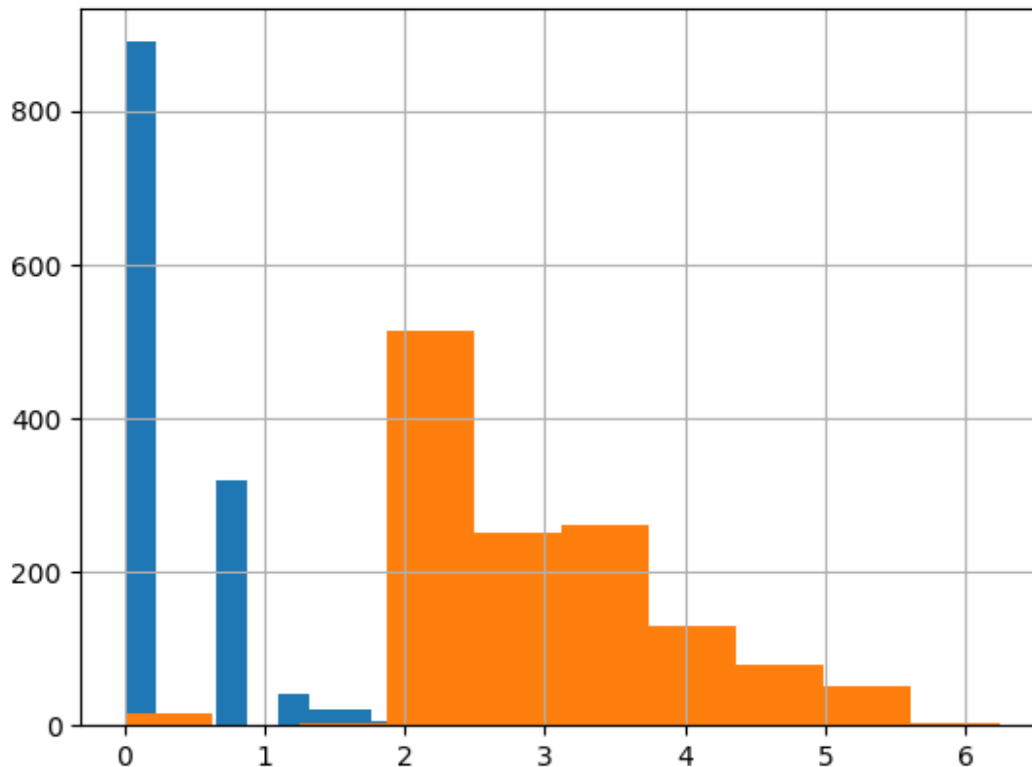
#create dummy variables from categories
all_dummies = pd.get_dummies(all_data[['Pclass', 'Sex', 'Age',
'SibSp', 'Parch', 'norm_fare', 'Embarked', 'cabin_adv',
'cabin_multiple', 'numeric_ticket', 'name_title', 'train_test']])

#Split to Train Test again
X_train = all_dummies[all_dummies.train_test ==
1].drop(['train_test'], axis = 1)
X_test = all_dummies[all_dummies.train_test == 0].drop(['train_test'],
axis = 1)

y_train = all_data[all_data.train_test==1].Survived
y_train.shape

(889,)

```

#Scaling/Normalizing the Dataset

```
from sklearn.preprocessing import StandardScaler
scale = StandardScaler()
all_dummies_scaled = all_dummies.copy()
all_dummies_scaled[['Age', 'SibSp', 'Parch', 'norm_fare']] =
scale.fit_transform(all_dummies_scaled[['Age', 'SibSp', 'Parch',
'norm_fare']])
all_dummies_scaled
```

#Splitting the scaled dataset back to Training and Testing

```
X_train_scaled = all_dummies_scaled[all_dummies_scaled.train_test ==
1].drop(['train_test'], axis = 1)
X_test_scaled = all_dummies_scaled[all_dummies_scaled.train_test ==
0].drop(['train_test'], axis = 1)
```

```
y_train = all_data[all_data.train_test==1].Survived
```

```
X_test_scaled.head(10) #Printing the first 10 rows of the scaled test
data
```

	Age	SibSp	Parch	norm_fare	cabin_multiple
numeric_ticket \					
0	0.364532	-0.479537	-0.445407	-0.826704	0
1					
1	1.336872	0.480272	-0.445407	-0.928672	0

1					
2	2.503681	-0.479537	-0.445407	-0.629214	0
1					
3	-0.218872	-0.479537	-0.445407	-0.733456	0
1					
4	-0.607808	0.480272	0.709647	-0.404079	0
1					
5	-1.230105	-0.479537	-0.445407	-0.674954	0
1					
6	0.014490	-0.479537	-0.445407	-0.850393	0
1					
7	-0.296659	0.480272	0.709647	0.437919	0
1					
8	-0.918957	-0.479537	-0.445407	-0.899466	0
1					
9	-0.685595	1.440081	-0.445407	0.255598	0
0					

	Pclass_1	Pclass_2	Pclass_3	Sex_female	...	name_title_Master	\
0	False	False	True	False	...	False	
1	False	False	True	True	...	False	
2	False	True	False	False	...	False	
3	False	False	True	False	...	False	
4	False	False	True	True	...	False	
5	False	False	True	False	...	False	
6	False	False	True	True	...	False	
7	False	True	False	False	...	False	
8	False	False	True	True	...	False	
9	False	False	True	False	...	False	

	name_title_Miss	name_title_Mlle	name_title_Mme	name_title_Mr	\
0	False	False	False	True	
1	False	False	False	False	
2	False	False	False	True	
3	False	False	False	True	
4	False	False	False	False	
5	False	False	False	True	
6	True	False	False	False	
7	False	False	False	True	
8	False	False	False	False	
9	False	False	False	True	

	name_title_Mrs	name_title_Ms	name_title_Rev	name_title_Sir	\
0	False	False	False	False	
1	True	False	False	False	
2	False	False	False	False	
3	False	False	False	False	
4	True	False	False	False	
5	False	False	False	False	
6	False	False	False	False	

7	False	False	False	False
8	True	False	False	False
9	False	False	False	False

	name_title_the Countess
0	False
1	False
2	False
3	False
4	False
5	False
6	False
7	False
8	False
9	False

[10 rows x 41 columns]

```
#Importing useful scikit-learn functions
from sklearn.model_selection import cross_val_score
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
from sklearn import tree
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
```

From here we'll calculate and compare the cross validation scores of these 7 ML models on our training data to select the model with the best score for training and prediction.

- GaussianNB()
- LogisticRegression()
- DecisionTreeClassifier()
- KNeighborsClassifier()
- RandomForestClassifier()
- SVC() (Support Vector Classifier)
- XGBClassifier() (Extreme Gradient Boost)

```
gnb = GaussianNB()
cv = cross_val_score(gnb, X_train_scaled, y_train, cv=5)
print(cv)
print(cv.mean())

[0.66853933 0.70224719 0.75842697 0.74719101 0.73446328]
0.7221735542436362

lr = LogisticRegression(max_iter = 2000)
cv = cross_val_score(lr, X_train_scaled, y_train, cv=5)
print(cv)
print(cv.mean())
```

```
[0.82022472 0.80898876 0.80337079 0.82022472 0.85310734]
0.8211832666793626
```

```
dt = tree.DecisionTreeClassifier(random_state = 1)
cv = cross_val_score(dt, X_train_scaled, y_train, cv=5)
print(cv)
print(cv.mean())
```

```
[0.74719101 0.74157303 0.80898876 0.75280899 0.81920904]
0.7739541674601662
```

```
knn = KNeighborsClassifier()
cv = cross_val_score(knn, X_train_scaled, y_train, cv=5)
print(cv)
print(cv.mean())
```

```
[0.79775281 0.79213483 0.83146067 0.80337079 0.85310734]
0.8155652891512728
```

```
rf = RandomForestClassifier(random_state = 1)
cv = cross_val_score(rf, X_train_scaled, y_train, cv=5)
print(cv)
print(cv.mean())
```

```
[0.81460674 0.78651685 0.85393258 0.73033708 0.84180791]
0.8054402336062972
```

```
svc = SVC(probability = True)
cv = cross_val_score(svc, X_train_scaled, y_train, cv=5)
print(cv)
print(cv.mean())
```

```
[0.84831461 0.82022472 0.8258427 0.80337079 0.86440678]
0.8324319177299563
```

```
from xgboost import XGBClassifier
xgb = XGBClassifier(random_state = 1)
cv = cross_val_score(xgb, X_train_scaled, y_train, cv=5)
print(cv)
print(cv.mean())
```

```
[0.81460674 0.79775281 0.85955056 0.79775281 0.81355932]
0.8166444486764426
```

Comparing the CV scores that we generated above the SVC model turns out to be the best one. Hence, we'll train our data using this model.

```
from sklearn.model_selection import GridSearchCV
```

```

svc = SVC(probability = True)
#setting up the hyperparameters for svc
param_grid = tuned_parameters = [{'kernel': ['rbf'], 'gamma': [.1, .5,
1, 2, 5, 10, 12], 'C': [.1, 1, 10, 100, 1000]},
{'kernel': ['linear'], 'C': [.1, 1,
10, 100, 1000]}],
{'kernel': ['poly'], 'degree': [2, 3,
4, 5, 6, 8], 'C': [.1, 1, 10, 100, 1000]}]

clf_svc = GridSearchCV(svc, param_grid = param_grid, cv=5, verbose =
True, n_jobs=-1)
#Training the model on scaled X_train and Y_train.
best_clf_svc = clf_svc.fit(X_train_scaled, y_train)
#Generating the accuracy score of the trained model (On Training
Dataset)
best_clf_svc.best_score_

```

Fitting 5 folds for each of 70 candidates, totalling 350 fits

0.8324319177299563

```

#Predicting the outcome of Test Data
y_hat_base_bsvc = best_clf_svc.predict(X_test_scaled).astype(int)
#Storing the outcome in a dataframe
submission = {'PassengerId': df_test.PassengerId, 'Survived':
y_hat_base_bsvc}
submission = pd.DataFrame(data=submission)
#Exporting/Submitting the dataframe to .csv
submission.to_csv('/kaggle/working/submission.csv', index=False)
submission.head()

```

	PassengerId	Survived
0	892	0
1	893	1
2	894	0
3	895	0
4	896	1

Attempting to improve the accuracy using XGBoost

```

xgb = XGBClassifier(
    random_state=42
)
param_grid = {
    'max_depth': [5, 7, 9, 11],
    'learning_rate': [0.005, 0.01, 0.1, 0.2],
    'n_estimators': [50, 100, 150],
    'subsample': [0.8, 0.9, 1],
    'colsample_bytree': [0.8, 0.9, 1]
}

```

```

}
clf_xgb = GridSearchCV(xgb, param_grid = param_grid, cv=5, verbose =
True, n_jobs=-1)

#Training the model on scaled X_train and Y_train.
best_clf_xgb = clf_xgb.fit(X_train_scaled, y_train)
#Generating the accuracy score of the trained model (On Training
Dataset)
best_clf_xgb.best_score_

Fitting 5 folds for each of 432 candidates, totalling 2160 fits

0.8436551767917223

#Predicting the outcome of Test Data
#y_hat_base_bxgb = best_clf_xgb.predict(X_test_scaled).astype(int)
#Storing the outcome in a dataframe
#submission = {'PassengerId': df_test.PassengerId, 'Survived':
y_hat_base_bxgb}
#submission = pd.DataFrame(data=submission)
#Exporting/Submitting the dataframe to .csv
#submission.to_csv('/kaggle/working/submission.csv', index=False)
#submission.head()

```

Attempting to improve the accuracy using Neural Network

```

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

# Set a random seed for reproducibility
tf.random.set_seed(42)

# Define the model architecture
model = Sequential([
    Dense(64, activation='relu'), # Input layer
    # Dropout(0.3),
    Dense(32, activation='relu'), # Hidden layer 1
    # Dropout(0.3), #Dropout layer
    Dense(1, activation='sigmoid') # Output layer
])

# Compile the model
model.compile(optimizer =
tf.keras.optimizers.Adam(learning_rate=0.002),
              loss='binary_crossentropy',
              metrics=['accuracy'])

# Assuming you have training data (X_train, y_train) and testing data

```

```

(X_test, y_test)
model.fit(X_train_scaled, y_train, epochs=300, batch_size=32)

# Evaluate the model on the test data
print('Train accuracy:', model.evaluate(X_train_scaled, y_train))

2024-12-28 10:07:22.858012: E
external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:9261] Unable
to register cuDNN factory: Attempting to register factory for plugin
cuDNN when one has already been registered
2024-12-28 10:07:22.858174: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:607] Unable to
register cuFFT factory: Attempting to register factory for plugin
cuFFT when one has already been registered
2024-12-28 10:07:23.028534: E
external/local_xla/xla/stream_executor/cuda/cuda_blas.cc:1515] Unable
to register cuBLAS factory: Attempting to register factory for plugin
cuBLAS when one has already been registered

Epoch 1/300
28/28 _____ 1s 2ms/step - accuracy: 0.5821 - loss:
0.6624
Epoch 2/300
28/28 _____ 0s 2ms/step - accuracy: 0.7779 - loss:
0.4720
Epoch 3/300
28/28 _____ 0s 2ms/step - accuracy: 0.8301 - loss:
0.4064
Epoch 4/300
28/28 _____ 0s 2ms/step - accuracy: 0.8412 - loss:
0.3884
Epoch 5/300
28/28 _____ 0s 2ms/step - accuracy: 0.8450 - loss:
0.3757
Epoch 6/300
28/28 _____ 0s 2ms/step - accuracy: 0.8565 - loss:
0.3675
Epoch 7/300
28/28 _____ 0s 2ms/step - accuracy: 0.8526 - loss:
0.3602
Epoch 8/300
28/28 _____ 0s 2ms/step - accuracy: 0.8522 - loss:
0.3539
Epoch 9/300
28/28 _____ 0s 2ms/step - accuracy: 0.8559 - loss:
0.3484
Epoch 10/300
28/28 _____ 0s 2ms/step - accuracy: 0.8612 - loss:
0.3436
Epoch 11/300

```

```
28/28 _____ 0s 2ms/step - accuracy: 0.8696 - loss:
0.3386
Epoch 12/300
28/28 _____ 0s 2ms/step - accuracy: 0.8681 - loss:
0.3342
Epoch 13/300
28/28 _____ 0s 2ms/step - accuracy: 0.8685 - loss:
0.3297
Epoch 14/300
28/28 _____ 0s 2ms/step - accuracy: 0.8706 - loss:
0.3248
Epoch 15/300
28/28 _____ 0s 2ms/step - accuracy: 0.8715 - loss:
0.3207
Epoch 16/300
28/28 _____ 0s 2ms/step - accuracy: 0.8700 - loss:
0.3169
Epoch 17/300
28/28 _____ 0s 2ms/step - accuracy: 0.8715 - loss:
0.3132
Epoch 18/300
28/28 _____ 0s 2ms/step - accuracy: 0.8722 - loss:
0.3104
Epoch 19/300
28/28 _____ 0s 2ms/step - accuracy: 0.8702 - loss:
0.3072
Epoch 20/300
28/28 _____ 0s 2ms/step - accuracy: 0.8750 - loss:
0.3038
Epoch 21/300
28/28 _____ 0s 2ms/step - accuracy: 0.8774 - loss:
0.3007
Epoch 22/300
28/28 _____ 0s 2ms/step - accuracy: 0.8769 - loss:
0.2986
Epoch 23/300
28/28 _____ 0s 2ms/step - accuracy: 0.8786 - loss:
0.2943
Epoch 24/300
28/28 _____ 0s 2ms/step - accuracy: 0.8780 - loss:
0.2925
Epoch 25/300
28/28 _____ 0s 2ms/step - accuracy: 0.8872 - loss:
0.2895
Epoch 26/300
28/28 _____ 0s 2ms/step - accuracy: 0.8862 - loss:
0.2872
Epoch 27/300
28/28 _____ 0s 2ms/step - accuracy: 0.8904 - loss:
```



```
0.2854
Epoch 28/300
28/28 _____ 0s 2ms/step - accuracy: 0.8890 - loss:
0.2836
Epoch 29/300
28/28 _____ 0s 2ms/step - accuracy: 0.8916 - loss:
0.2809
Epoch 30/300
28/28 _____ 0s 2ms/step - accuracy: 0.8933 - loss:
0.2799
Epoch 31/300
28/28 _____ 0s 2ms/step - accuracy: 0.8954 - loss:
0.2768
Epoch 32/300
28/28 _____ 0s 2ms/step - accuracy: 0.8946 - loss:
0.2749
Epoch 33/300
28/28 _____ 0s 2ms/step - accuracy: 0.8976 - loss:
0.2731
Epoch 34/300
28/28 _____ 0s 2ms/step - accuracy: 0.8970 - loss:
0.2726
Epoch 35/300
28/28 _____ 0s 2ms/step - accuracy: 0.8982 - loss:
0.2704
Epoch 36/300
28/28 _____ 0s 2ms/step - accuracy: 0.8976 - loss:
0.2674
Epoch 37/300
28/28 _____ 0s 2ms/step - accuracy: 0.8970 - loss:
0.2681
Epoch 38/300
28/28 _____ 0s 2ms/step - accuracy: 0.8991 - loss:
0.2654
Epoch 39/300
28/28 _____ 0s 2ms/step - accuracy: 0.8998 - loss:
0.2637
Epoch 40/300
28/28 _____ 0s 2ms/step - accuracy: 0.8994 - loss:
0.2625
Epoch 41/300
28/28 _____ 0s 2ms/step - accuracy: 0.9015 - loss:
0.2612
Epoch 42/300
28/28 _____ 0s 2ms/step - accuracy: 0.9001 - loss:
0.2584
Epoch 43/300
28/28 _____ 0s 2ms/step - accuracy: 0.9035 - loss:
0.2586
```

```
Epoch 44/300
28/28 _____ 0s 2ms/step - accuracy: 0.9010 - loss:
0.2581
Epoch 45/300
28/28 _____ 0s 2ms/step - accuracy: 0.9004 - loss:
0.2558
Epoch 46/300
28/28 _____ 0s 2ms/step - accuracy: 0.9010 - loss:
0.2548
Epoch 47/300
28/28 _____ 0s 2ms/step - accuracy: 0.9044 - loss:
0.2541
Epoch 48/300
28/28 _____ 0s 2ms/step - accuracy: 0.9042 - loss:
0.2537
Epoch 49/300
28/28 _____ 0s 2ms/step - accuracy: 0.9030 - loss:
0.2514
Epoch 50/300
28/28 _____ 0s 2ms/step - accuracy: 0.9042 - loss:
0.2516
Epoch 51/300
28/28 _____ 0s 2ms/step - accuracy: 0.9035 - loss:
0.2488
Epoch 52/300
28/28 _____ 0s 2ms/step - accuracy: 0.9040 - loss:
0.2479
Epoch 53/300
28/28 _____ 0s 2ms/step - accuracy: 0.9043 - loss:
0.2486
Epoch 54/300
28/28 _____ 0s 2ms/step - accuracy: 0.9041 - loss:
0.2456
Epoch 55/300
28/28 _____ 0s 2ms/step - accuracy: 0.9041 - loss:
0.2445
Epoch 56/300
28/28 _____ 0s 2ms/step - accuracy: 0.9037 - loss:
0.2445
Epoch 57/300
28/28 _____ 0s 2ms/step - accuracy: 0.9041 - loss:
0.2425
Epoch 58/300
28/28 _____ 0s 2ms/step - accuracy: 0.9041 - loss:
0.2426
Epoch 59/300
28/28 _____ 0s 2ms/step - accuracy: 0.9047 - loss:
0.2423
Epoch 60/300
```

```
28/28 ————— 0s 2ms/step - accuracy: 0.9054 - loss:
0.2410
Epoch 61/300
28/28 ————— 0s 2ms/step - accuracy: 0.9041 - loss:
0.2410
Epoch 62/300
28/28 ————— 0s 2ms/step - accuracy: 0.9034 - loss:
0.2391
Epoch 63/300
28/28 ————— 0s 2ms/step - accuracy: 0.9041 - loss:
0.2389
Epoch 64/300
28/28 ————— 0s 2ms/step - accuracy: 0.9033 - loss:
0.2400
Epoch 65/300
28/28 ————— 0s 2ms/step - accuracy: 0.9069 - loss:
0.2370
Epoch 66/300
28/28 ————— 0s 2ms/step - accuracy: 0.9041 - loss:
0.2356
Epoch 67/300
28/28 ————— 0s 2ms/step - accuracy: 0.9075 - loss:
0.2355
Epoch 68/300
28/28 ————— 0s 2ms/step - accuracy: 0.9062 - loss:
0.2343
Epoch 69/300
28/28 ————— 0s 2ms/step - accuracy: 0.9072 - loss:
0.2343
Epoch 70/300
28/28 ————— 0s 2ms/step - accuracy: 0.9051 - loss:
0.2324
Epoch 71/300
28/28 ————— 0s 2ms/step - accuracy: 0.9068 - loss:
0.2319
Epoch 72/300
28/28 ————— 0s 2ms/step - accuracy: 0.9081 - loss:
0.2324
Epoch 73/300
28/28 ————— 0s 2ms/step - accuracy: 0.9048 - loss:
0.2336
Epoch 74/300
28/28 ————— 0s 2ms/step - accuracy: 0.9085 - loss:
0.2315
Epoch 75/300
28/28 ————— 0s 2ms/step - accuracy: 0.9079 - loss:
0.2308
Epoch 76/300
28/28 ————— 0s 2ms/step - accuracy: 0.9087 - loss:
```

```
0.2293
Epoch 77/300
28/28 _____ 0s 2ms/step - accuracy: 0.9116 - loss:
0.2282
Epoch 78/300
28/28 _____ 0s 2ms/step - accuracy: 0.9063 - loss:
0.2274
Epoch 79/300
28/28 _____ 0s 2ms/step - accuracy: 0.9064 - loss:
0.2285
Epoch 80/300
28/28 _____ 0s 1ms/step - accuracy: 0.9146 - loss:
0.2260
Epoch 81/300
28/28 _____ 0s 2ms/step - accuracy: 0.9118 - loss:
0.2274
Epoch 82/300
28/28 _____ 0s 2ms/step - accuracy: 0.9088 - loss:
0.2253
Epoch 83/300
28/28 _____ 0s 2ms/step - accuracy: 0.9061 - loss:
0.2242
Epoch 84/300
28/28 _____ 0s 2ms/step - accuracy: 0.9060 - loss:
0.2275
Epoch 85/300
28/28 _____ 0s 2ms/step - accuracy: 0.9118 - loss:
0.2261
Epoch 86/300
28/28 _____ 0s 2ms/step - accuracy: 0.9085 - loss:
0.2255
Epoch 87/300
28/28 _____ 0s 2ms/step - accuracy: 0.9087 - loss:
0.2266
Epoch 88/300
28/28 _____ 0s 2ms/step - accuracy: 0.9071 - loss:
0.2245
Epoch 89/300
28/28 _____ 0s 2ms/step - accuracy: 0.9084 - loss:
0.2238
Epoch 90/300
28/28 _____ 0s 2ms/step - accuracy: 0.9098 - loss:
0.2228
Epoch 91/300
28/28 _____ 0s 2ms/step - accuracy: 0.9066 - loss:
0.2231
Epoch 92/300
28/28 _____ 0s 2ms/step - accuracy: 0.9084 - loss:
0.2229
```

```
Epoch 93/300
28/28 _____ 0s 2ms/step - accuracy: 0.9113 - loss:
0.2229
Epoch 94/300
28/28 _____ 0s 2ms/step - accuracy: 0.9158 - loss:
0.2218
Epoch 95/300
28/28 _____ 0s 2ms/step - accuracy: 0.9146 - loss:
0.2205
Epoch 96/300
28/28 _____ 0s 2ms/step - accuracy: 0.9108 - loss:
0.2208
Epoch 97/300
28/28 _____ 0s 1ms/step - accuracy: 0.9144 - loss:
0.2200
Epoch 98/300
28/28 _____ 0s 2ms/step - accuracy: 0.9114 - loss:
0.2201
Epoch 99/300
28/28 _____ 0s 2ms/step - accuracy: 0.9108 - loss:
0.2196
Epoch 100/300
28/28 _____ 0s 2ms/step - accuracy: 0.9075 - loss:
0.2189
Epoch 101/300
28/28 _____ 0s 2ms/step - accuracy: 0.9132 - loss:
0.2186
Epoch 102/300
28/28 _____ 0s 2ms/step - accuracy: 0.9109 - loss:
0.2187
Epoch 103/300
28/28 _____ 0s 2ms/step - accuracy: 0.9157 - loss:
0.2164
Epoch 104/300
28/28 _____ 0s 2ms/step - accuracy: 0.9095 - loss:
0.2186
Epoch 105/300
28/28 _____ 0s 2ms/step - accuracy: 0.9146 - loss:
0.2165
Epoch 106/300
28/28 _____ 0s 2ms/step - accuracy: 0.9090 - loss:
0.2172
Epoch 107/300
28/28 _____ 0s 2ms/step - accuracy: 0.9123 - loss:
0.2170
Epoch 108/300
28/28 _____ 0s 2ms/step - accuracy: 0.9105 - loss:
0.2148
Epoch 109/300
```

```
28/28 ————— 0s 2ms/step - accuracy: 0.9128 - loss:
0.2153
Epoch 110/300
28/28 ————— 0s 2ms/step - accuracy: 0.9153 - loss:
0.2163
Epoch 111/300
28/28 ————— 0s 2ms/step - accuracy: 0.9131 - loss:
0.2153
Epoch 112/300
28/28 ————— 0s 2ms/step - accuracy: 0.9120 - loss:
0.2141
Epoch 113/300
28/28 ————— 0s 2ms/step - accuracy: 0.9164 - loss:
0.2143
Epoch 114/300
28/28 ————— 0s 2ms/step - accuracy: 0.9093 - loss:
0.2147
Epoch 115/300
28/28 ————— 0s 2ms/step - accuracy: 0.9176 - loss:
0.2123
Epoch 116/300
28/28 ————— 0s 2ms/step - accuracy: 0.9156 - loss:
0.2123
Epoch 117/300
28/28 ————— 0s 2ms/step - accuracy: 0.9129 - loss:
0.2127
Epoch 118/300
28/28 ————— 0s 2ms/step - accuracy: 0.9117 - loss:
0.2135
Epoch 119/300
28/28 ————— 0s 2ms/step - accuracy: 0.9161 - loss:
0.2131
Epoch 120/300
28/28 ————— 0s 2ms/step - accuracy: 0.9152 - loss:
0.2108
Epoch 121/300
28/28 ————— 0s 2ms/step - accuracy: 0.9146 - loss:
0.2111
Epoch 122/300
28/28 ————— 0s 2ms/step - accuracy: 0.9179 - loss:
0.2109
Epoch 123/300
28/28 ————— 0s 2ms/step - accuracy: 0.9141 - loss:
0.2096
Epoch 124/300
28/28 ————— 0s 2ms/step - accuracy: 0.9197 - loss:
0.2103
Epoch 125/300
28/28 ————— 0s 2ms/step - accuracy: 0.9161 - loss:
```

```
0.2093
Epoch 126/300
28/28 _____ 0s 2ms/step - accuracy: 0.9168 - loss:
0.2091
Epoch 127/300
28/28 _____ 0s 2ms/step - accuracy: 0.9168 - loss:
0.2096
Epoch 128/300
28/28 _____ 0s 2ms/step - accuracy: 0.9165 - loss:
0.2085
Epoch 129/300
28/28 _____ 0s 2ms/step - accuracy: 0.9208 - loss:
0.2076
Epoch 130/300
28/28 _____ 0s 2ms/step - accuracy: 0.9174 - loss:
0.2070
Epoch 131/300
28/28 _____ 0s 2ms/step - accuracy: 0.9172 - loss:
0.2075
Epoch 132/300
28/28 _____ 0s 2ms/step - accuracy: 0.9169 - loss:
0.2086
Epoch 133/300
28/28 _____ 0s 2ms/step - accuracy: 0.9164 - loss:
0.2066
Epoch 134/300
28/28 _____ 0s 2ms/step - accuracy: 0.9188 - loss:
0.2057
Epoch 135/300
28/28 _____ 0s 2ms/step - accuracy: 0.9106 - loss:
0.2059
Epoch 136/300
28/28 _____ 0s 2ms/step - accuracy: 0.9226 - loss:
0.2056
Epoch 137/300
28/28 _____ 0s 2ms/step - accuracy: 0.9197 - loss:
0.2060
Epoch 138/300
28/28 _____ 0s 2ms/step - accuracy: 0.9211 - loss:
0.2055
Epoch 139/300
28/28 _____ 0s 1ms/step - accuracy: 0.9214 - loss:
0.2050
Epoch 140/300
28/28 _____ 0s 2ms/step - accuracy: 0.9163 - loss:
0.2026
Epoch 141/300
28/28 _____ 0s 2ms/step - accuracy: 0.9171 - loss:
0.2036
```

```
Epoch 142/300
28/28 _____ 0s 2ms/step - accuracy: 0.9212 - loss:
0.2028
Epoch 143/300
28/28 _____ 0s 2ms/step - accuracy: 0.9187 - loss:
0.2042
Epoch 144/300
28/28 _____ 0s 2ms/step - accuracy: 0.9185 - loss:
0.2034
Epoch 145/300
28/28 _____ 0s 2ms/step - accuracy: 0.9181 - loss:
0.2025
Epoch 146/300
28/28 _____ 0s 2ms/step - accuracy: 0.9154 - loss:
0.2024
Epoch 147/300
28/28 _____ 0s 2ms/step - accuracy: 0.9216 - loss:
0.2011
Epoch 148/300
28/28 _____ 0s 2ms/step - accuracy: 0.9212 - loss:
0.2013
Epoch 149/300
28/28 _____ 0s 2ms/step - accuracy: 0.9197 - loss:
0.2011
Epoch 150/300
28/28 _____ 0s 2ms/step - accuracy: 0.9199 - loss:
0.2010
Epoch 151/300
28/28 _____ 0s 2ms/step - accuracy: 0.9150 - loss:
0.2015
Epoch 152/300
28/28 _____ 0s 2ms/step - accuracy: 0.9216 - loss:
0.2006
Epoch 153/300
28/28 _____ 0s 2ms/step - accuracy: 0.9180 - loss:
0.2009
Epoch 154/300
28/28 _____ 0s 2ms/step - accuracy: 0.9189 - loss:
0.2002
Epoch 155/300
28/28 _____ 0s 2ms/step - accuracy: 0.9166 - loss:
0.2013
Epoch 156/300
28/28 _____ 0s 2ms/step - accuracy: 0.9204 - loss:
0.1994
Epoch 157/300
28/28 _____ 0s 2ms/step - accuracy: 0.9200 - loss:
0.2016
Epoch 158/300
```



```
28/28 ————— 0s 2ms/step - accuracy: 0.9188 - loss:
0.2016
Epoch 159/300
28/28 ————— 0s 2ms/step - accuracy: 0.9169 - loss:
0.2003
Epoch 160/300
28/28 ————— 0s 2ms/step - accuracy: 0.9213 - loss:
0.1989
Epoch 161/300
28/28 ————— 0s 2ms/step - accuracy: 0.9157 - loss:
0.1981
Epoch 162/300
28/28 ————— 0s 2ms/step - accuracy: 0.9209 - loss:
0.1991
Epoch 163/300
28/28 ————— 0s 2ms/step - accuracy: 0.9170 - loss:
0.1987
Epoch 164/300
28/28 ————— 0s 2ms/step - accuracy: 0.9210 - loss:
0.1984
Epoch 165/300
28/28 ————— 0s 2ms/step - accuracy: 0.9196 - loss:
0.1977
Epoch 166/300
28/28 ————— 0s 2ms/step - accuracy: 0.9202 - loss:
0.1976
Epoch 167/300
28/28 ————— 0s 2ms/step - accuracy: 0.9215 - loss:
0.1964
Epoch 168/300
28/28 ————— 0s 2ms/step - accuracy: 0.9134 - loss:
0.1977
Epoch 169/300
28/28 ————— 0s 2ms/step - accuracy: 0.9208 - loss:
0.1972
Epoch 170/300
28/28 ————— 0s 2ms/step - accuracy: 0.9201 - loss:
0.1959
Epoch 171/300
28/28 ————— 0s 2ms/step - accuracy: 0.9156 - loss:
0.1970
Epoch 172/300
28/28 ————— 0s 2ms/step - accuracy: 0.9181 - loss:
0.1970
Epoch 173/300
28/28 ————— 0s 2ms/step - accuracy: 0.9201 - loss:
0.1960
Epoch 174/300
28/28 ————— 0s 2ms/step - accuracy: 0.9199 - loss:
0.1962
```

```
Epoch 175/300
28/28 _____ 0s 2ms/step - accuracy: 0.9164 - loss:
0.1953
Epoch 176/300
28/28 _____ 0s 2ms/step - accuracy: 0.9183 - loss:
0.1956
Epoch 177/300
28/28 _____ 0s 2ms/step - accuracy: 0.9193 - loss:
0.1955
Epoch 178/300
28/28 _____ 0s 2ms/step - accuracy: 0.9144 - loss:
0.1936
Epoch 179/300
28/28 _____ 0s 2ms/step - accuracy: 0.9184 - loss:
0.1947
Epoch 180/300
28/28 _____ 0s 2ms/step - accuracy: 0.9193 - loss:
0.1961
Epoch 181/300
28/28 _____ 0s 2ms/step - accuracy: 0.9188 - loss:
0.1938
Epoch 182/300
28/28 _____ 0s 2ms/step - accuracy: 0.9203 - loss:
0.1933
Epoch 183/300
28/28 _____ 0s 2ms/step - accuracy: 0.9129 - loss:
0.1931
Epoch 184/300
28/28 _____ 0s 2ms/step - accuracy: 0.9176 - loss:
0.1949
Epoch 185/300
28/28 _____ 0s 2ms/step - accuracy: 0.9203 - loss:
0.1943
Epoch 186/300
28/28 _____ 0s 2ms/step - accuracy: 0.9231 - loss:
0.1944
Epoch 187/300
28/28 _____ 0s 2ms/step - accuracy: 0.9134 - loss:
0.1934
Epoch 188/300
28/28 _____ 0s 2ms/step - accuracy: 0.9160 - loss:
0.1931
Epoch 189/300
28/28 _____ 0s 2ms/step - accuracy: 0.9146 - loss:
0.1917
Epoch 190/300
28/28 _____ 0s 2ms/step - accuracy: 0.9184 - loss:
0.1922
Epoch 191/300
```

```
28/28 ————— 0s 2ms/step - accuracy: 0.9096 - loss:
0.1927
Epoch 192/300
28/28 ————— 0s 2ms/step - accuracy: 0.9206 - loss:
0.1921
Epoch 193/300
28/28 ————— 0s 2ms/step - accuracy: 0.9162 - loss:
0.1905
Epoch 194/300
28/28 ————— 0s 2ms/step - accuracy: 0.9195 - loss:
0.1910
Epoch 195/300
28/28 ————— 0s 2ms/step - accuracy: 0.9198 - loss:
0.1905
Epoch 196/300
28/28 ————— 0s 2ms/step - accuracy: 0.9158 - loss:
0.1903
Epoch 197/300
28/28 ————— 0s 1ms/step - accuracy: 0.9200 - loss:
0.1900
Epoch 198/300
28/28 ————— 0s 2ms/step - accuracy: 0.9136 - loss:
0.1902
Epoch 199/300
28/28 ————— 0s 2ms/step - accuracy: 0.9225 - loss:
0.1903
Epoch 200/300
28/28 ————— 0s 2ms/step - accuracy: 0.9071 - loss:
0.1908
Epoch 201/300
28/28 ————— 0s 2ms/step - accuracy: 0.9197 - loss:
0.1890
Epoch 202/300
28/28 ————— 0s 2ms/step - accuracy: 0.9146 - loss:
0.1911
Epoch 203/300
28/28 ————— 0s 2ms/step - accuracy: 0.9184 - loss:
0.1887
Epoch 204/300
28/28 ————— 0s 2ms/step - accuracy: 0.9178 - loss:
0.1896
Epoch 205/300
28/28 ————— 0s 1ms/step - accuracy: 0.9158 - loss:
0.1892
Epoch 206/300
28/28 ————— 0s 2ms/step - accuracy: 0.9167 - loss:
0.1878
Epoch 207/300
28/28 ————— 0s 2ms/step - accuracy: 0.9178 - loss:
```

```
0.1880
Epoch 208/300
28/28 _____ 0s 2ms/step - accuracy: 0.9190 - loss:
0.1884
Epoch 209/300
28/28 _____ 0s 2ms/step - accuracy: 0.9194 - loss:
0.1873
Epoch 210/300
28/28 _____ 0s 2ms/step - accuracy: 0.9162 - loss:
0.1869
Epoch 211/300
28/28 _____ 0s 2ms/step - accuracy: 0.9157 - loss:
0.1877
Epoch 212/300
28/28 _____ 0s 2ms/step - accuracy: 0.9234 - loss:
0.1885
Epoch 213/300
28/28 _____ 0s 2ms/step - accuracy: 0.9184 - loss:
0.1858
Epoch 214/300
28/28 _____ 0s 2ms/step - accuracy: 0.9190 - loss:
0.1862
Epoch 215/300
28/28 _____ 0s 2ms/step - accuracy: 0.9210 - loss:
0.1867
Epoch 216/300
28/28 _____ 0s 2ms/step - accuracy: 0.9216 - loss:
0.1874
Epoch 217/300
28/28 _____ 0s 2ms/step - accuracy: 0.9188 - loss:
0.1861
Epoch 218/300
28/28 _____ 0s 2ms/step - accuracy: 0.9219 - loss:
0.1860
Epoch 219/300
28/28 _____ 0s 2ms/step - accuracy: 0.9142 - loss:
0.1861
Epoch 220/300
28/28 _____ 0s 2ms/step - accuracy: 0.9210 - loss:
0.1857
Epoch 221/300
28/28 _____ 0s 2ms/step - accuracy: 0.9197 - loss:
0.1846
Epoch 222/300
28/28 _____ 0s 2ms/step - accuracy: 0.9204 - loss:
0.1858
Epoch 223/300
28/28 _____ 0s 1ms/step - accuracy: 0.9178 - loss:
0.1855
```

```
Epoch 224/300
28/28 _____ 0s 2ms/step - accuracy: 0.9153 - loss:
0.1845
Epoch 225/300
28/28 _____ 0s 2ms/step - accuracy: 0.9183 - loss:
0.1857
Epoch 226/300
28/28 _____ 0s 2ms/step - accuracy: 0.9218 - loss:
0.1854
Epoch 227/300
28/28 _____ 0s 2ms/step - accuracy: 0.9115 - loss:
0.1845
Epoch 228/300
28/28 _____ 0s 2ms/step - accuracy: 0.9236 - loss:
0.1860
Epoch 229/300
28/28 _____ 0s 2ms/step - accuracy: 0.9129 - loss:
0.1844
Epoch 230/300
28/28 _____ 0s 2ms/step - accuracy: 0.9226 - loss:
0.1836
Epoch 231/300
28/28 _____ 0s 2ms/step - accuracy: 0.9178 - loss:
0.1849
Epoch 232/300
28/28 _____ 0s 2ms/step - accuracy: 0.9259 - loss:
0.1852
Epoch 233/300
28/28 _____ 0s 2ms/step - accuracy: 0.9155 - loss:
0.1833
Epoch 234/300
28/28 _____ 0s 2ms/step - accuracy: 0.9202 - loss:
0.1822
Epoch 235/300
28/28 _____ 0s 2ms/step - accuracy: 0.9147 - loss:
0.1854
Epoch 236/300
28/28 _____ 0s 2ms/step - accuracy: 0.9226 - loss:
0.1841
Epoch 237/300
28/28 _____ 0s 2ms/step - accuracy: 0.9131 - loss:
0.1838
Epoch 238/300
28/28 _____ 0s 1ms/step - accuracy: 0.9231 - loss:
0.1827
Epoch 239/300
28/28 _____ 0s 2ms/step - accuracy: 0.9163 - loss:
0.1827
Epoch 240/300
```

```
28/28 ————— 0s 2ms/step - accuracy: 0.9212 - loss: 0.1845
Epoch 241/300
28/28 ————— 0s 2ms/step - accuracy: 0.9165 - loss: 0.1849
Epoch 242/300
28/28 ————— 0s 2ms/step - accuracy: 0.9164 - loss: 0.1825
Epoch 243/300
28/28 ————— 0s 2ms/step - accuracy: 0.9186 - loss: 0.1825
Epoch 244/300
28/28 ————— 0s 2ms/step - accuracy: 0.9233 - loss: 0.1811
Epoch 245/300
28/28 ————— 0s 2ms/step - accuracy: 0.9214 - loss: 0.1811
Epoch 246/300
28/28 ————— 0s 2ms/step - accuracy: 0.9231 - loss: 0.1802
Epoch 247/300
28/28 ————— 0s 2ms/step - accuracy: 0.9148 - loss: 0.1810
Epoch 248/300
28/28 ————— 0s 2ms/step - accuracy: 0.9228 - loss: 0.1810
Epoch 249/300
28/28 ————— 0s 2ms/step - accuracy: 0.9190 - loss: 0.1807
Epoch 250/300
28/28 ————— 0s 2ms/step - accuracy: 0.9280 - loss: 0.1793
Epoch 251/300
28/28 ————— 0s 2ms/step - accuracy: 0.9184 - loss: 0.1810
Epoch 252/300
28/28 ————— 0s 2ms/step - accuracy: 0.9243 - loss: 0.1799
Epoch 253/300
28/28 ————— 0s 1ms/step - accuracy: 0.9277 - loss: 0.1839
Epoch 254/300
28/28 ————— 0s 2ms/step - accuracy: 0.9231 - loss: 0.1824
Epoch 255/300
28/28 ————— 0s 2ms/step - accuracy: 0.9250 - loss: 0.1792
Epoch 256/300
28/28 ————— 0s 2ms/step - accuracy: 0.9245 - loss:
```

```
0.1783
Epoch 257/300
28/28 _____ 0s 2ms/step - accuracy: 0.9237 - loss:
0.1814
Epoch 258/300
28/28 _____ 0s 2ms/step - accuracy: 0.9215 - loss:
0.1823
Epoch 259/300
28/28 _____ 0s 2ms/step - accuracy: 0.9231 - loss:
0.1806
Epoch 260/300
28/28 _____ 0s 2ms/step - accuracy: 0.9181 - loss:
0.1797
Epoch 261/300
28/28 _____ 0s 2ms/step - accuracy: 0.9229 - loss:
0.1788
Epoch 262/300
28/28 _____ 0s 2ms/step - accuracy: 0.9149 - loss:
0.1792
Epoch 263/300
28/28 _____ 0s 2ms/step - accuracy: 0.9222 - loss:
0.1784
Epoch 264/300
28/28 _____ 0s 2ms/step - accuracy: 0.9137 - loss:
0.1828
Epoch 265/300
28/28 _____ 0s 2ms/step - accuracy: 0.9210 - loss:
0.1804
Epoch 266/300
28/28 _____ 0s 2ms/step - accuracy: 0.9180 - loss:
0.1794
Epoch 267/300
28/28 _____ 0s 2ms/step - accuracy: 0.9210 - loss:
0.1785
Epoch 268/300
28/28 _____ 0s 2ms/step - accuracy: 0.9237 - loss:
0.1779
Epoch 269/300
28/28 _____ 0s 2ms/step - accuracy: 0.9184 - loss:
0.1768
Epoch 270/300
28/28 _____ 0s 2ms/step - accuracy: 0.9225 - loss:
0.1762
Epoch 271/300
28/28 _____ 0s 2ms/step - accuracy: 0.9229 - loss:
0.1771
Epoch 272/300
28/28 _____ 0s 2ms/step - accuracy: 0.9253 - loss:
0.1762
```

```
Epoch 273/300
28/28 _____ 0s 2ms/step - accuracy: 0.9248 - loss:
0.1753
Epoch 274/300
28/28 _____ 0s 2ms/step - accuracy: 0.9265 - loss:
0.1746
Epoch 275/300
28/28 _____ 0s 1ms/step - accuracy: 0.9226 - loss:
0.1758
Epoch 276/300
28/28 _____ 0s 1ms/step - accuracy: 0.9254 - loss:
0.1756
Epoch 277/300
28/28 _____ 0s 2ms/step - accuracy: 0.9156 - loss:
0.1750
Epoch 278/300
28/28 _____ 0s 2ms/step - accuracy: 0.9261 - loss:
0.1746
Epoch 279/300
28/28 _____ 0s 2ms/step - accuracy: 0.9227 - loss:
0.1757
Epoch 280/300
28/28 _____ 0s 2ms/step - accuracy: 0.9218 - loss:
0.1747
Epoch 281/300
28/28 _____ 0s 2ms/step - accuracy: 0.9211 - loss:
0.1747
Epoch 282/300
28/28 _____ 0s 1ms/step - accuracy: 0.9225 - loss:
0.1742
Epoch 283/300
28/28 _____ 0s 2ms/step - accuracy: 0.9205 - loss:
0.1746
Epoch 284/300
28/28 _____ 0s 2ms/step - accuracy: 0.9240 - loss:
0.1744
Epoch 285/300
28/28 _____ 0s 2ms/step - accuracy: 0.9239 - loss:
0.1752
Epoch 286/300
28/28 _____ 0s 2ms/step - accuracy: 0.9237 - loss:
0.1728
Epoch 287/300
28/28 _____ 0s 2ms/step - accuracy: 0.9235 - loss:
0.1734
Epoch 288/300
28/28 _____ 0s 2ms/step - accuracy: 0.9237 - loss:
0.1741
Epoch 289/300
```



```

28/28 ————— 0s 2ms/step - accuracy: 0.9217 - loss:
0.1749
Epoch 290/300
28/28 ————— 0s 2ms/step - accuracy: 0.9204 - loss:
0.1746
Epoch 291/300
28/28 ————— 0s 2ms/step - accuracy: 0.9262 - loss:
0.1717
Epoch 292/300
28/28 ————— 0s 1ms/step - accuracy: 0.9194 - loss:
0.1723
Epoch 293/300
28/28 ————— 0s 2ms/step - accuracy: 0.9258 - loss:
0.1724
Epoch 294/300
28/28 ————— 0s 2ms/step - accuracy: 0.9259 - loss:
0.1714
Epoch 295/300
28/28 ————— 0s 2ms/step - accuracy: 0.9287 - loss:
0.1719
Epoch 296/300
28/28 ————— 0s 2ms/step - accuracy: 0.9233 - loss:
0.1712
Epoch 297/300
28/28 ————— 0s 2ms/step - accuracy: 0.9243 - loss:
0.1743
Epoch 298/300
28/28 ————— 0s 2ms/step - accuracy: 0.9246 - loss:
0.1716
Epoch 299/300
28/28 ————— 0s 1ms/step - accuracy: 0.9188 - loss:
0.1731
Epoch 300/300
28/28 ————— 0s 2ms/step - accuracy: 0.9224 - loss:
0.1726
28/28 ————— 0s 1ms/step - accuracy: 0.9253 - loss:
0.1681
Train accuracy: [0.1570606827735901, 0.9347581267356873]

```

```

#Predicting Outcome of Test Data
#y_hat_nn = model.predict(X_test_scaled).flatten().astype(int)
#Storing the outcome in a dataframe
#submission = {'PassengerId': df_test.PassengerId, 'Survived':
y_hat_nn}
#submission = pd.DataFrame(data=submission)
#Exporting/Submitting the dataframe to .csv
#submission.to_csv('/kaggle/working/submission.csv', index=False)

```

Conclusion

The best Test Accuracy Score achieved is 0.77990 using SVC model. Two attempts to improve this test accuracy score using XGBoost Classifier model and Sequential Neural Network failed.

